

programare ce extinde limbajului C, folosit pentru dezvoltarea de aplicații iOS și macOS. Swift este succesorul limbajului Objective-C.

O caracteristică importantă a Xcode este editorul de interfețe grafice integrat (storyboardul), care permite generarea vizuală a interfețelor grafice și conectarea mai multor ferestre fără necesitatea de a scrie cod.

În ceea ce privește editorul pentru scrierea codului sursă, oricare din editoarele descrise mai sus suportă un număr mare de limbaje de programare și este adaptat pentru acestea. Recomandarea noastră este să încercați mai multe editoare și să alegeți pe cel pe care îl considerați cel mai comod în folosire, iar pentru limbajele de programare care au un IDE dedicat, se recomandă folosirea IDE-ului.

6.6.5 Biblioteci și framework-uri

Un alt aspect de care trebuie să ținem cont în dezvoltarea programelor este folosirea bibliotecilor și a framework-urilor.

Bibliotecile sunt colecții de resurse pe care le integrăm în aplicațiile noastre pentru a ne ușura procesul de dezvoltare. De exemplu, dacă dorim să dezvoltăm o aplicație Python care face cereri HTTP, vom folosi biblioteca *requests* care expune funcția *get()* pentru a face o cerere. Fără a folosi biblioteca, ar trebui să scriem noi tot codul care se conectează la un server, generează pachetul pentru cerere, îl trimite, apoi preia și interpretează răspunsul.

O unealtă asemănătoare bibliotecilor sunt framework-urile. Și acestea sunt unelte care ușurează procesul de dezvoltare, dar ele oferă un schelet pe care aplicația îl urmează pentru a beneficia de proprietățile framework-ului. Practic putem să ne gândim că biblioteca e un modul care odată integrat în aplicație aduce funcții sau constante noi pe care le putem folosi când e cazul, în timp ce framework-ul impune anumite funcții sau elemente ce trebuie implementate în aplicație și aplicația trebuie să se muleze după acea structură.

Pentru a putea integra biblioteci și framework-uri în aplicațiile pe care le dezvoltăm, acestea trebuie să existe pe sistemul pe care lucrăm. Vom vedea mai departe, că de multe ori fiecare platformă de dezvoltare specifică unui limbaj vine cu propriul sistem de instalare a acestor module.

6.6.6 Rularea programului

Odată ce am scris codul sursă și am instalat bibliotecile sau framework-urile folosite, pasul final este să rulăm programul. Complexitatea acestui pas depinde de complexitatea programului obținut. Pentru un program simplu, cu un fișier care conține codul sursă și care nu folosește nicio bibliotecă, în general, este suficient să specificăm fișierul sursă și compilatorul sau interpretorul se ocupă de restul procesului. Listing 6.1 conține comanda de compilare și de rulare a unui program simplu.

```
1 student@uso:~$ gcc main.c -o main
2 student@uso:~$ ./main
```

Listing 6.1: Compilarea și rularea unui program simplu

În cazul unui program complex care conține mai multe fișiere, vom rula mai multe comenzi, ca în Listing 6.2. Linia 1 și linia 2 conțin comenzi de compilare, linia 3 este comanda de linking, iar linia 4 este comanda de rulare.

```
1 student@uso:~$ gcc -c utils.c
2 student@uso:~$ gcc -c main.c
3 student@uso:~$ gcc utils.o main.o -o main
4 student@uso:~$ ./main
```

Listing 6.2: Compilarea și rularea unui program din mai multe surse

6.6.7 Automatizarea procesului de dezvoltare

Cum am observat mai sus, pentru aplicații complexe, compilarea sau rularea acestora constă în rularea unei secvențe de comenzi. Toate aceste comenzi trebuie rulate indiferent dacă se aduc modificări majore programului, sau dacă se schimbă doar o linie de cod. Astfel, nu e greu să ne dăm seama că avem nevoie de o modalitate de a automatiza acțiunile necesare compilării și rulării codului, astfel încât să rulăm o comandă simplă care să *se ocupe de tot*. Practic, putem să ne gândim că fișierele care alcătuiesc programul pe care l-am dezvoltat sunt ingredientele necesare pentru o prăjitură, iar prăjitura este executabilul sau chiar rezultatul rulării programului. Astfel, similar unei rețete care descrie pașii necesari pentru a obține prăjitura, vom scrie un fișier care conține pașii necesari obținerii rezultatului programului, iar utilitarul în cauză este bucătarul care urmează rețeta.

Unele dintre utilitarele cele mai folosite pentru automatizarea acestor taskuri sunt Make, Grunt, Gulp.

6.7 Medii de dezvoltare

Pentru fiecare limbaj de programare folosit avem nevoie de mediul de dezvoltare aferent (aplicații, biblioteci, compilator și/sau interpretor), pentru a putea rula aplicațiile pe care le scriem. Bineînțeles că putem scrie codul sursă în orice editor, fără a folosi nici un alt program adiament, dar am obține un fișier cu instrucțiuni inutile, pe care nu îl putem compila sau interpreta. De aceea, în primul rând, în funcție de tipul limbajului, avem nevoie de un compilator sau un interpretor, care să ne permită acest lucru.

Pe lângă compilator sau interpretor, pentru folosirea unui limbaj de programare avem nevoie să instalăm bibliotecile folosite.

Mai departe vom analiza particularități în instalarea compilatorului sau a interpretorului pentru mai multe limbaje de programare.

6.7.1 C / C++

Pentru limbajul C, compilatorul cel mai cunoscut este GCC (*GNU Compiler Collection*). Conceput inițial pentru a compila limbajul C, în momentul de față GCC a fost extins pentru a suporta compilarea și a altor limbaje, cum ar fi C++ sau Java.

Pentru sistemele derivate din Debian, există un pachet special numit *build-essential*, instalabil folosind comanda:

```
1 student@uso:~$ sudo apt install build-essential
```

Pentru sistemele derivate din Redat, precum Fedora, CentOS etc. trebuie instalate pachetele separat folosind comanda:

```
1 student@uso:~$ sudo dnf install make gcc g++
```

Toate aceste pachete vor instala și limbajul C++.

GCC poate fi instalat și pe sisteme Windows, prin intermediul unei aplicații care simulează un sistem Unix (Cygwin, MinGW etc.).

Pentru sisteme Windows, compilatorul de baza este integrat în aplicația Microsoft Visual C++. Microsoft Visual C++ este un ecosistem de programare și constă într-un editor cu funcții avansate (sugestii de completare a codului, sugestii de posibile erori etc.) împreună cu compilatorul de C și C++ și se recomandă folosirea acestuia pentru dezvoltarea de programe C.

Pentru C++ se poate folosi aceeași colecție de compilatoare ca pentru limbajul C, adică GCC. Cum am menționat mai sus, în momentul de față GCC suportă compilarea limbajului C++.

La folosirea **gcc**, sistemul analizează fișierele de intrare și hotărăște dacă limbajul compilat este C sau C++. Pentru a eficientiza procesul, se poate folosi direct **g++**, adică compilatorul de C++ integrat în GCC, ca în Listing 6.3.

```
1 student@uso:~$ g++ main.cpp -o main
2 student@uso:~$ ./main
```

Listing 6.3: Exemplu compilare și rulare C++

De asemenea, pentru sisteme Windows mediul de dezvoltare recomandat rămâne același. Microsoft Visual C++ este recomandat și pentru dezvoltarea de aplicații C++.

6.7.1.1 Dezvoltarea în C/C++

Cum am menționat anterior, compilatorul cel mai folosit pentru C și C++ este gcc/g++. Pentru folosirea compilatorului este suficient să rulăm comanda **gcc** cu parametrii aferenți.

În procesul de compilare codul sursă este prelucrat și trece prin mai multe etape intermediare:

1. preprocesare: se înlocuiesc macro-uri, se numerotează fiecare linie de cod; rezultatul e un fișier cu extensia **.i**
2. compilare: codul sursă este transformat în cod în limbaj de asamblare; rezultatul e un fișier cu extensia **.s**
3. asamblare: codul în limbaj de asamblare este transformat în cod mașină; rezultatul e un fișier cu extensia **.o**
4. link-editare: se fac legăturile către fișiere externe care conțin simboluri sau funcții apelate din fișierul sursă; dacă avem o funcție definită într-un fișier și folosită în altul, implementarea funcției trebuie legată de apelul ei;

La simpla rulare a comenzii **gcc**, se trece prin toate etapele menționate. De exemplu, pentru un program C simplu (`main.c` fără nicio bibliotecă) vom rula comenzile indicate în Listing 6.1.

În schimb, pentru programe mai complexe, alcătuite din mai multe fișiere, e necesar să generăm fișierul obiect aferent fiecărui fișier sursă și apoi să le link-edităm pentru a obține executabilul final, așa cum am indicat în Listing 6.2.

6.7.1.2 Automatizarea compilării în C/C++

Cum am menționat mai devreme, este obositor să rulăm toate comenzile de mai sus pentru fiecare modificare adusă în codul sursă. De aceea există posibilitatea de a crea un fișier care conține toate aceste comenzi și care le poate rula pe toate printr-un singur apel.

Utilitarul de automatizare cel mai folosit pentru aplicațiile C/C++ este utilitarul Make. Pentru a folosi Make e suficient să creăm un fișier cu numele `Makefile` în structura programului nostru. La rularea comenzii **make**, utilitarul găsește acel fișier și execută instrucțiunile descrise.

Un fișier `Makefile` urmează formatul din Listing 6.4.

```
1 regula: dependente
2 <TAB> comanda
```

Listing 6.4: Formatul unui fișier Makefile

Fișierul poate să conțină mai multe astfel de linii, precum fișierul `Makefile` din Listing 6.5.

```
1 build: utils.o hello.o help.o
2     gcc utils.o help.o hello.o -o hello
3
4 all:
5     gcc simple_hello.c -o simple
6
7 utils.o: utils.c
8     gcc -c utils.c
9
10 hello.o: hello.c
11     gcc -c hello.c
12
13 help.o: help.c
14     gcc -c help.c
15
16 clean:
17     rm -f *.o hello
```

Listing 6.5: Exemplu Makefile

Regula reprezintă numele unei instrucțiuni. La simpla rulare a comenzii **make** prima regulă din fișier este cea care va fi executată. În schimb, dacă vom scrie **make clean**, se va executa comanda aferentă regulii `clean`.

Dependențele sunt fișiere și/sau reguli necesare pentru a putea rula o comandă. Practic, la rularea unei comenzi se verifică dacă fișierul din dependență există. Dacă acesta există, putem rula comanda, dacă nu, se caută o altă regulă cu acel nume și se va rula

acea comandă (dacă dependențele ei sunt îndeplinite, dacă nu, se continuă lanțul de dependențe). Bineînțeles, sunt cazuri în care dependențele pot să lipsească.

Acum să luăm cazul programului dezvoltat mai sus, care depinde de mai multe fișiere și să presupunem că am schimbat o linie într-un fișier. La rularea comenzii **make**, toate fișierele ar fi refăcute, ceea ce nu este eficient. De aceea, **make** ține cont de încă un aspect la verificarea dependențelor. Dacă dependența (fișierul) există, **make** va urmări lanțul de dependențe până ajunge la fișierele de bază ale acestora. Dacă fișierele au fost modificate după ce dependența a fost creată, aceasta va fi regenerată, în caz contrar nu se va regenera dependența.

6.7.2 Java

Fiind un limbaj hibrid, pentru Java sistemul de execuție a codului sursă este mai complex. Cum am menționat mai sus, în general limbajele hibride trec printr-o etapă de compilare în urma căreia rezultă un fișier bytecode, iar apoi acel cod este interpretat.

Pentru majoritatea limbajelor hibride (Python, JavaScript) există un sistem care conține atât compilatorul de bytecode cât și interpretorul, sau mașina virtuală. Pentru Java, cele două elemente sunt separate clar, avem compilatorul care generează codul sursă în bytecode și mașina virtuală ce execută fișierele bytecode. Cele două se instalează separat, ca două aplicații diferite și independente. Astfel, putem avea un sistem care doar generează fișierele bytecode și un alt sistem care le execută.

În ceea ce privește platformele pentru a compila și a rula Java, putem alege între două platforme principale: OracleJDK sau OpenJDK. Cum sugerează și numele, OracleJDK este dezvoltată de Oracle, în timp ce OpenJDK este o platformă open source și este special concepută pentru sisteme Unix. Pe lângă JDK (Java Development Kit), putem instala OracleJRE sau OpenJRE (Java Runtime Environment).

În timp ce JDK este sistemul complet ce compilează și execută aplicații Java, JRE este doar mașina virtuală care rulează bytecode-ul generat de către compilator. Ea este inclusă în JDK, dar poate fi instalată și separat. Astfel, putem avea sisteme care doar rulează aplicații Java și care necesită bytecode-ul generat de JDK.

Pe sisteme Linux OpenJDK este într-un pachet cu numele de forma `openjdk-X-jdk`, unde `X` este numele versiunii. De exemplu, pe Ubuntu 18.04 pachetul este `openjdk-8-jdk`.

Pentru sisteme care rulează Windows, se recomandă instalarea OracleJDK. Deși se găsesc versiuni OpenJDK compatibile cu Windows, acestea nu sunt întreținute și prezintă riscuri de securitate.

Pentru a instala OracleJDK pe un sistem Windows, accesați site-ul Oracle¹ și urmați instrucțiunile.

O particularitate a Java este aceea că bibliotecile sunt împachetate ca niște arhive (jar - *Java Archive*) care sunt incluse în aplicațiile pe care le dezvoltăm. Fișierul `.jar` conține resurse diverse, de la definiții de clase la imagini sau alte fișiere multimedia. Fiecare astfel de arhivă poate conține un fișier manifest ce oferă informații despre pachet și despre utilizarea sa.

¹<https://www.oracle.com/technetwork/java/javase/overview/install-windows-142126.html>